

SECURITY CONSULTANT ENGAGEMENT

DevSecOps Pipeline Implementation

Acme Financial Services | Vasu Alli

Agenda

1 Risk Brief

5 minutes — business framing before building anything

2 Live Demo: Pipeline in Action

15 minutes — SAST, secrets scanning, SCA, container scanning

3 Rollout Plan

10 minutes — phased approach for 40 microservices

4 Trade-offs & What I'd Do Differently

10 minutes + Q&A buffer

Current State: Why This Matters

- CI/CD with zero security scanning
- Secrets committed in .env files
- No dependency or container vulnerability management
- No branch protection — direct merges to main
- All AWS IAM roles: AdministratorAccess
- No WAF, no DDoS protection, permissive CORS (*)

Top 5 Risks (Day 1 Risk Brief)

#	Risk	Rating
1	Secrets committed to Git	CRITICAL
2	AWS IAM AdministratorAccess on all roles	CRITICAL
3	No code review before merge	HIGH
4	No dependency / vulnerability management	HIGH
5	Permissive CORS (*)	MEDIUM

Fix first: #1 + #2 together — both describe a state we may already be compromised in, not just at risk.

Pipeline Architecture — Five Parallel Gates

#	Gate	Tool / Method	Threshold
1	SAST	Semgrep + 2 custom rules + OWASP rulesets	Gate on Critical/High
2	Secrets	Gitleaks	Block any detected secret
3	SCA + SBOM	npm audit + Syft (CycloneDX)	Gate on High/Critical CVEs
4	Container	Trivy on hardened Dockerfile	Gate on Critical/High
5	Branch Protection	Required PR + approval + status checks	Preventive control

All five run in parallel on every push and PR — each reports independently, merge blocked until all pass.

Live Demo

SAST: Custom Rules on Real Juice Shop Code

The screenshot shows a GitHub Actions workflow run for a Semgrep Scan. The workflow is titled "Semgrep Scan" and is located at `github.com/vasuhacks7/juice-shop/actions/runs/30153446937/job/89667575137`. The status is "failed 12 minutes ago in 48s".

The workflow steps are:

- Run Semgrep (gate on Critical/High)

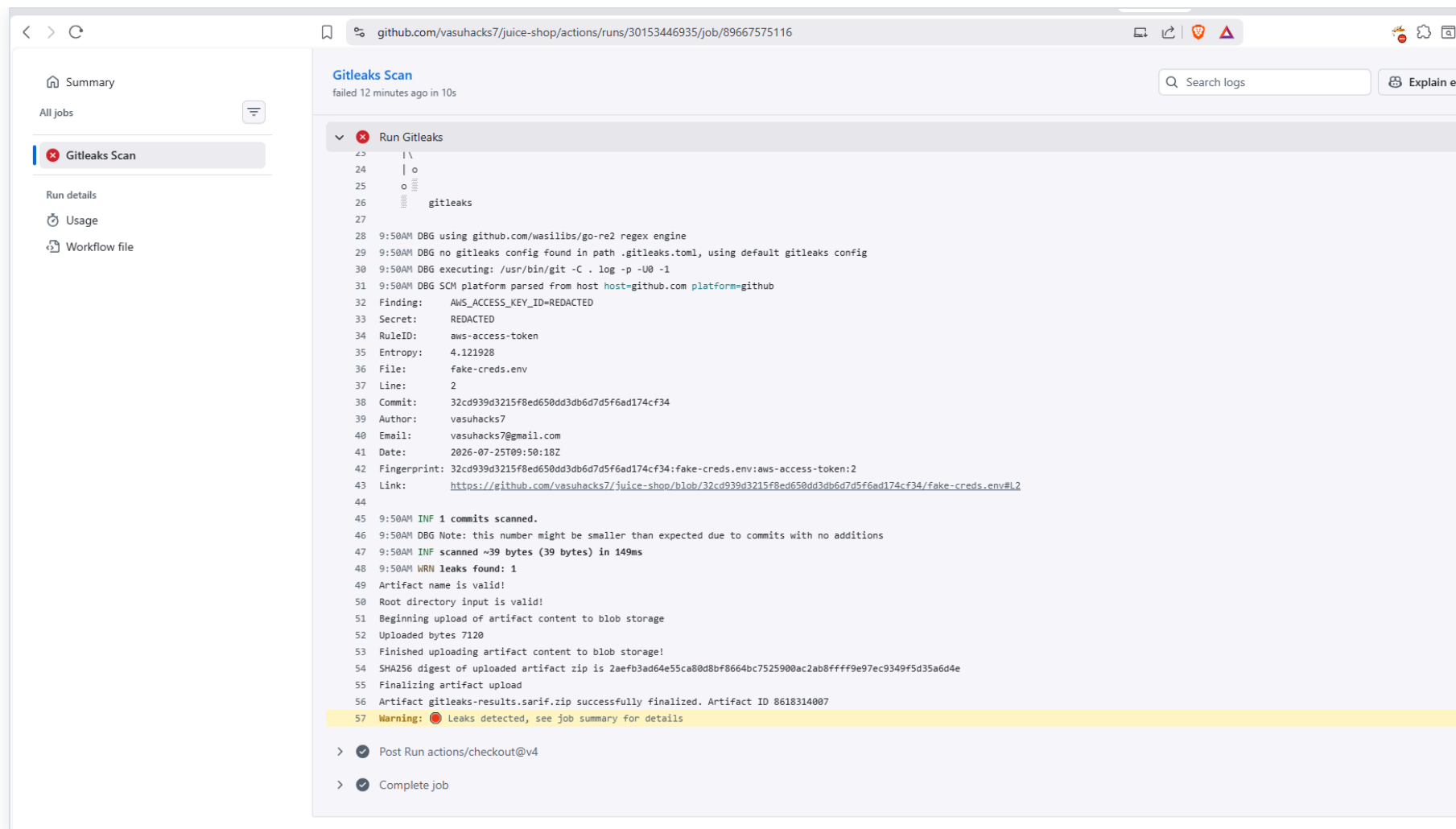
The output of the Semgrep Scan is as follows:

```
149 environment variable or a secret manager - anyone with read access to the repo can forge signatures
150 or tokens using this value.
151
152 18| return crypto.createHmac("sha256", "pa4qacea4VK9t9nGv7yZtwmj").update(data).digest("hex");
153 ;;-----
154 28| return jwt.sign(payload, "supersecret123");
155
156 >>> semgrep-rules.dangerous-eval-usage
157 (( Blocking ))
158 Use of eval() can execute arbitrary code if any part of the input is influenced by user-controlled
159 data. Avoid eval() entirely; use JSON.parse for data, or refactor to avoid dynamic code execution.
160
161 38| return eval(userInput);
162
163
164
165
166 | Scan Summary |
167
168 [X] Scan completed successfully.
169 • Findings: 15 (15 blocking)
170 • Rules run: 43
171 • Targets scanned: 995
172 • Parsed lines: ~100.0%
173 • Scan skipped:
174   • Files larger than 1.0 MB: 2
175   • Files matching .semgrepignore patterns: 155
176 • Scan was limited to files tracked by git
177 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
178 Ran 43 rules on 995 files: 15 findings.
179 Error: Process completed with exit code 1.
```

The workflow also includes the following steps:

- Upload results as artifact
- Post Run actions/checkout@v4
- Stop containers
- Complete job

Secrets Scanning: Gitleaks Catches a Fake AWS Key



Gitleaks Scan
failed 12 minutes ago in 10s

Search logs Explain e

Run Gitleaks

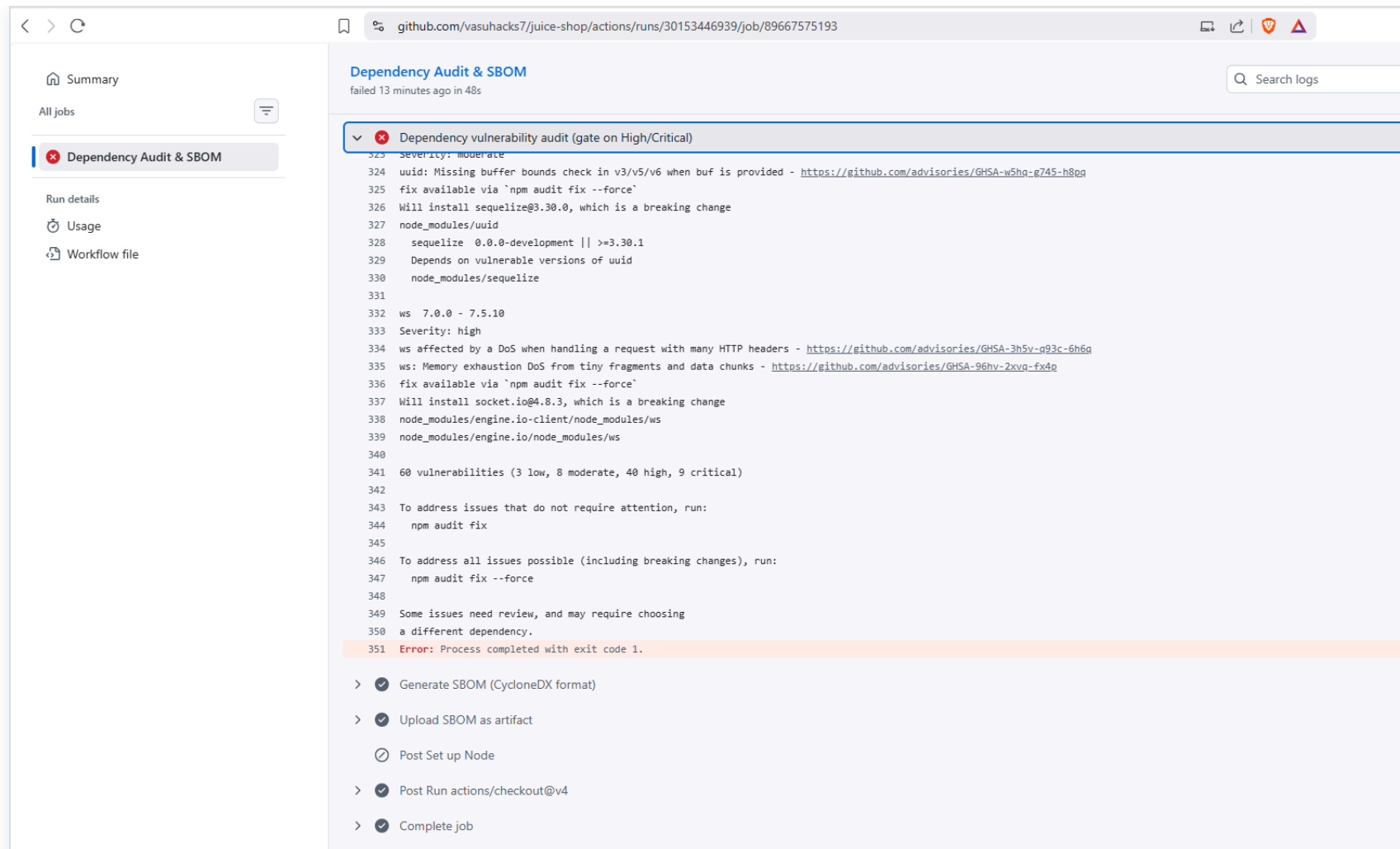
```
23 | \
24 | o
25 | o
26 | gitleaks
27 |
28 | 9:50AM DBG using github.com/wasilibs/go-re2 regex engine
29 | 9:50AM DBG no gitleaks config found in path .gitleaks.toml, using default gitleaks config
30 | 9:50AM DBG executing: /usr/bin/git -C . log -p -U0 -1
31 | 9:50AM DBG SCM platform parsed from host host=github.com platform=github
32 | Finding: AWS_ACCESS_KEY_ID=REDACTED
33 | Secret: REDACTED
34 | RuleID: aws-access-token
35 | Entropy: 4.121928
36 | File: fake-creds.env
37 | Line: 2
38 | Commit: 32cd939d3215f8ed658dd3db6d7d5f6ad174cf34
39 | Author: vasuhacks7
40 | Email: vasuhacks7@gmail.com
41 | Date: 2026-07-25T09:50:18Z
42 | Fingerprint: 32cd939d3215f8ed658dd3db6d7d5f6ad174cf34:fake-creds.env:aws-access-token:2
43 | Link: https://github.com/vasuhacks7/juice-shop/blob/32cd939d3215f8ed658dd3db6d7d5f6ad174cf34/fake-creds.env#L2
44 |
45 | 9:50AM INF 1 commits scanned.
46 | 9:50AM DBG Note: this number might be smaller than expected due to commits with no additions
47 | 9:50AM INF scanned ~39 bytes (39 bytes) in 149ms
48 | 9:50AM WRN leaks found: 1
49 | Artifact name is valid!
50 | Root directory input is valid!
51 | Beginning upload of artifact content to blob storage
52 | Uploaded bytes 7120
53 | Finished uploading artifact content to blob storage!
54 | SHA256 digest of uploaded artifact zip is 2aefb3ad64e55ca80d8bf8664bc7525900ac2ab8ffff9e97ec9349f5d35a6d4e
55 | Finalizing artifact upload
56 | Artifact gitleaks-results.sarif.zip successfully finalized. Artifact ID 8618314007
57 | Warning: Leaks detected, see job summary for details
```

> Post Run actions/checkout@v4

> Complete job

Rule: aws-access-token · File: fake-creds.env · dummy credential, not a real key

SCA: 60 Vulnerabilities, 40 High + 9 Critical



```
323 Severity: moderate
324 uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-w5hq-g745-h8qg
325 fix available via `npm audit fix --force`
326 Will install sequelize@3.30.0, which is a breaking change
327 node_modules/uuid
328   sequelize 0.0.0-development || >=3.30.1
329   Depends on vulnerable versions of uuid
330   node_modules/sequelize
331
332 ws 7.0.0 - 7.5.10
333 Severity: high
334 ws affected by a DoS when handling a request with many HTTP headers - https://github.com/advisories/GHSA-3h5v-g93c-6h6g
335 ws: Memory exhaustion DoS from tiny fragments and data chunks - https://github.com/advisories/GHSA-96hv-2xvq-fx4q
336 fix available via `npm audit fix --force`
337 Will install socket.io@4.8.3, which is a breaking change
338 node_modules/engine.io-client/node_modules/ws
339 node_modules/engine.io/node_modules/ws
340
341 60 vulnerabilities (3 low, 8 moderate, 40 high, 9 critical)
342
343 To address issues that do not require attention, run:
344   npm audit fix
345
346 To address all issues possible (including breaking changes), run:
347   npm audit fix --force
348
349 Some issues need review, and may require choosing
350 a different dependency.
351 Error: Process completed with exit code 1.
```

> ☒ Generate SBOM (CycloneDX format)

> ☒ Upload SBOM as artifact

☐ Post Set up Node

> ☒ Post Run actions/checkout@v4

> ☒ Complete job

DEPENDENCY AUDIT

60

total vulnerabilities

9 critical
40 high

SBOM generated in CycloneDX format
— an inventory, not just a scan result.

Container Scanning: Trivy on Hardened Image

The screenshot displays a GitHub Actions workflow run titled "Build & Scan Hardened Image" which failed 13 minutes ago. The left sidebar shows the workflow steps: "Scan Original Dockerfile (informational, non-blocking)" and "Build & Scan Hardened Image" (which failed). The main panel shows the logs for the failed step, "Scan image with Trivy (gate on Critical/High)". The logs indicate that the scan found a high-risk vulnerability: "Asymmetric Private Key (private-key)". The error message states: "Error: Process completed with exit code 1." The logs also show the command used to run Trivy: "Run rm -f trivy_envs.txt".

Build & Scan Hardened Image
failed 13 minutes ago in 2m 46s

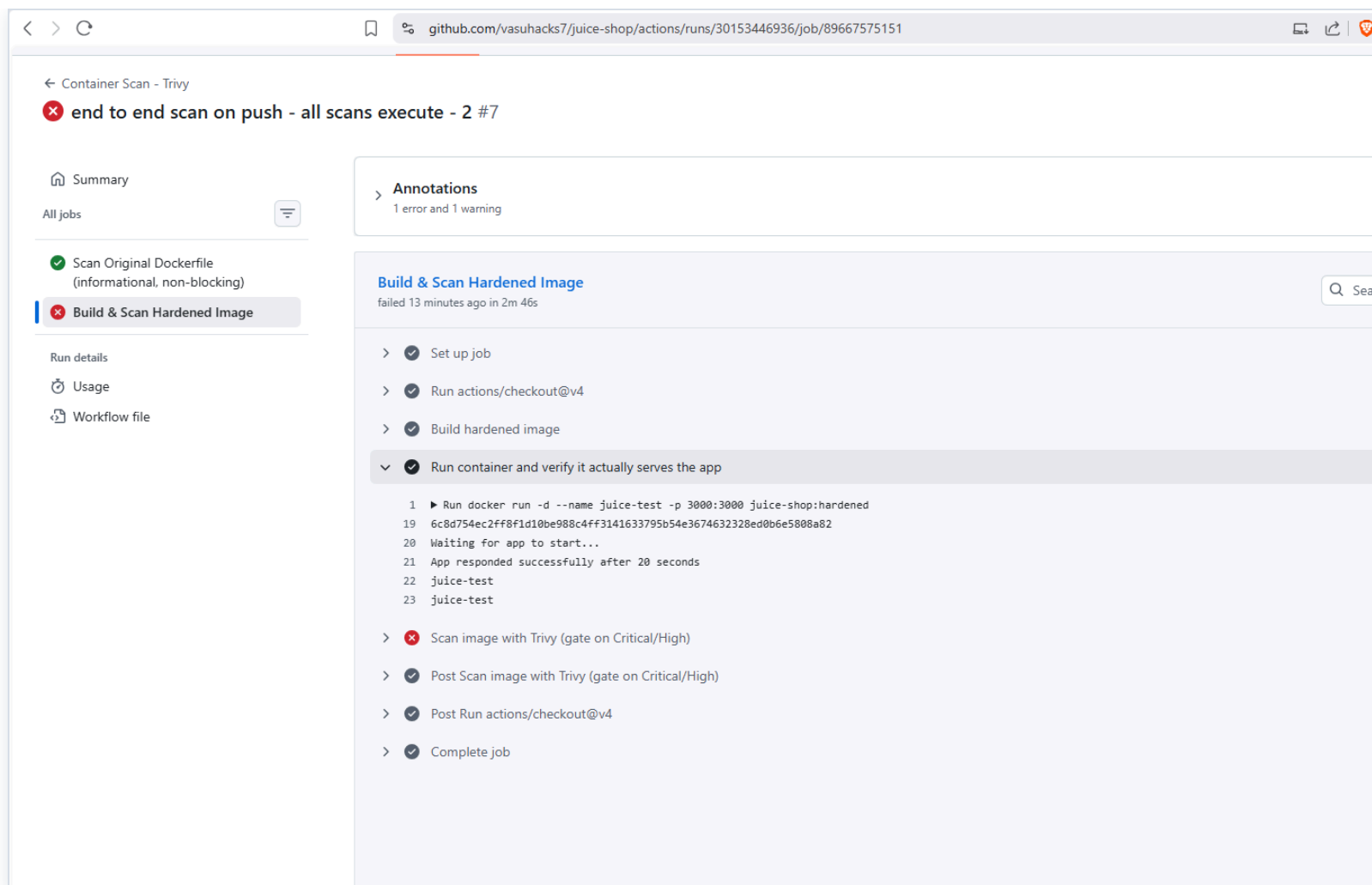
Search logs Explain error

Scan image with Trivy (gate on Critical/High) 9s

```
2062 /juice-shop/Makefile:2 (offset: 37 bytes) (added by 'COPY --chown=65532:0 /juice-shop . # bui')
2063
2064 1 AWS_ACCESS_KEY_ID=*****
2065 2 [ AWS_ACCESS_KEY_ID=*****
2066 3
2067
2068
2069
2070
2071 /juice-shop/lib/insecurity.ts (secrets)
2072 =====
2073 Total: 1 (HIGH: 1, CRITICAL: 0)
2074
2075 HIGH: AsymmetricPrivateKey (private-key)
2076 =====
2077 Asymmetric Private Key
2078
2079 /juice-shop/lib/insecurity.ts:21 (offset: 778 bytes) (added by 'COPY --chown=65532:0 /juice-shop . # bui')
2080
2081 19
2082 20 export const publicKey = fs ? fs.readFileSync('encryptionkeys/jwt.pub', 'utf8') : 'placeholder-publi
2083 21 [ -----BEGIN RSA PRIVATE KEY-----
2084
2085
2086
2087
2088 Error: Process completed with exit code 1.
2089   Run rm -f trivy_envs.txt
2090
2091
2092
```

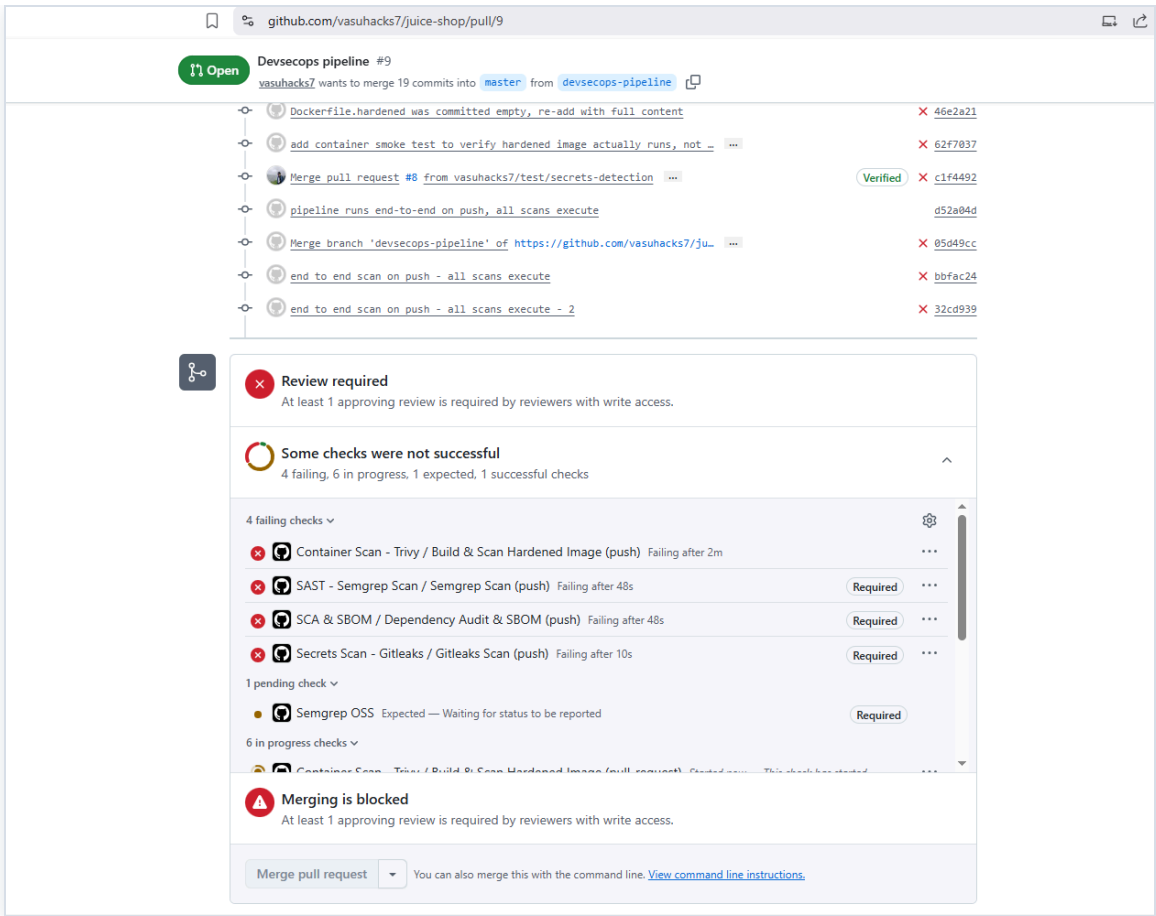
> ✓ Post Scan image with Trivy (gate on Critical/High) 0s
> ✓ Post Run actions/checkout@v4 0s
> ✓ Complete job 0s

Hardened Container: Builds AND Runs Successfully



docker run + smoke test — app responded successfully after 20 seconds

Branch Protection: Merge Blocked, Even for the Owner

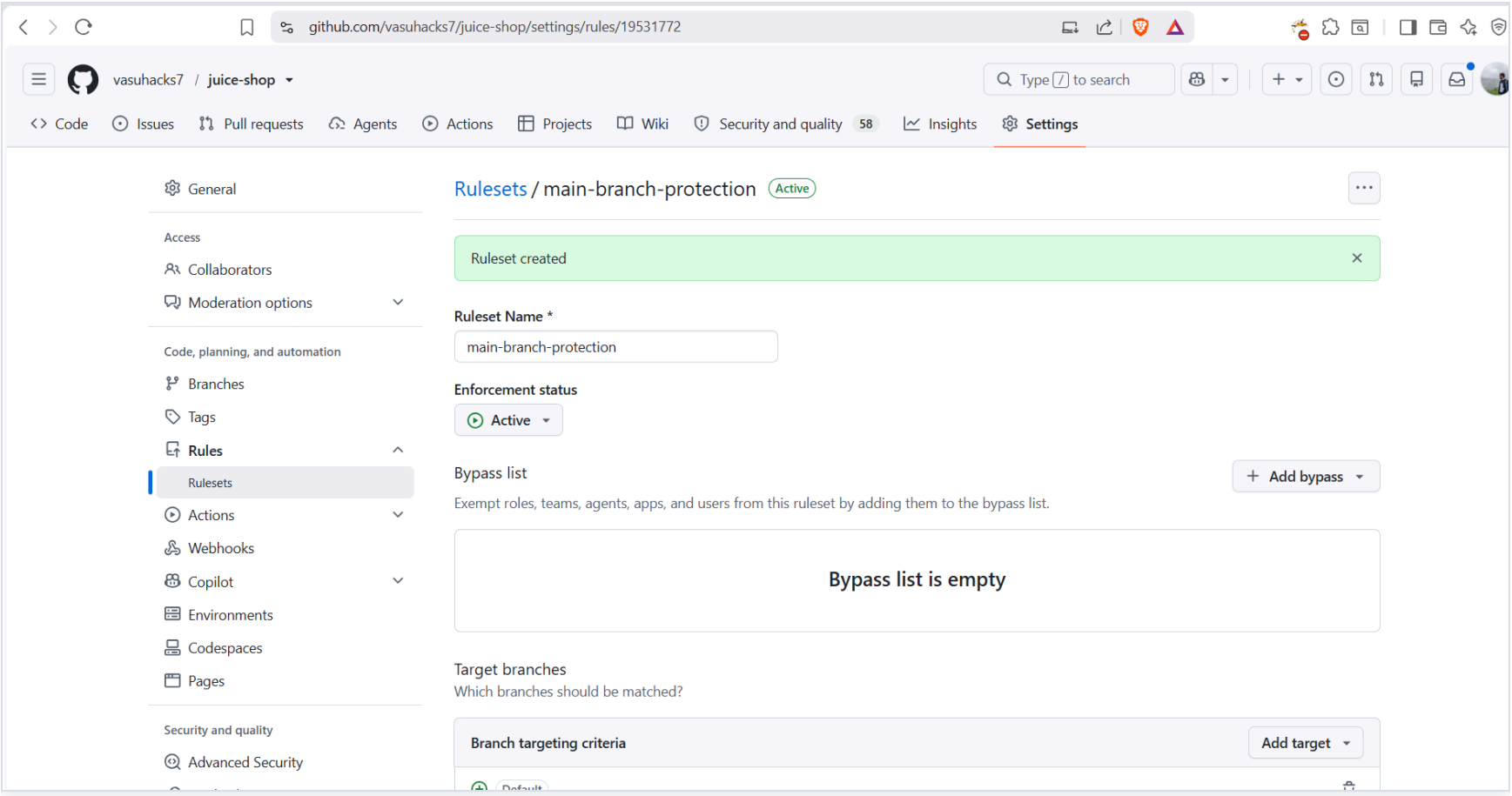


PR #9: 4 failing checks, review required, merging blocked

```
vasu.alli_wesecureap@LP-S0-TEK219 MINGW64 /d/juice-shop-devsecops/juice-shop (master)
$ git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 369 bytes | 369.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote: error: GH013: Repository rule violations found for refs/heads/master.
remote: Review all repository rules at https://github.com/vasuhacks7/juice-shop/rules?ref=refs%2Fheads%2Fmaster
remote:
remote: - Changes must be made through a pull request.
remote:
remote: - 2 of 2 required status checks are expected.
remote:
To https://github.com/vasuhacks7/juice-shop
 ! [remote rejected] master -> master (push declined due to repository rule violations)
error: failed to push some refs to 'https://github.com/vasuhacks7/juice-shop'
```

Direct push to master: GH013 rejection

Branch Protection: Ruleset Configuration



Rollout Plan

Rollout: Defining "Critical" + Three Phases

Criterion	Why
Handles money movement	Direct financial / regulatory exposure
Handles auth or PII	Fraud / identity theft risk
Internet-facing, no gateway	Directly reachable by an outside attacker

Phase	Timeline	Scope	Mode
1 — Pilot	Wk 1-2	3-5 highest-risk services	Warn-only
2 — Expand	Wk 3-6	All critical + internet-facing	Pilot→block, rest warn
3 — Full	Wk 7-12	Remaining ~30 services	SAST/secrets block org-wide

Start narrow on purpose — a small pilot surfaces process problems before they're multiplied across 40.

Adoption Strategy & Exception Handling

Metrics Tracked

- % of services onboarded (of 40)
- Mean time-to-fix for Critical/High
- Exceptions: requested vs. granted vs. expired
- Developer-reported false-positive rate

Exception Process

- Request in the PR itself
- Single-reviewer approval (same-day)
- Time-boxed, auto-expiring
- Escalate to EM on repetition

If most PRs on a service are requesting exceptions, that's a signal the rollout moved too fast — slow down rather than approve through it.

Trade-offs & Decisions

- Parallel gate execution (fast feedback) vs. sequential/staged (less wasted compute)
- Semgrep vs. CodeQL — chose Semgrep for faster setup and prior familiarity
- Dropped SARIF/GitHub Code Scanning UI for plain-text Actions logs — simpler, more reliable to demo
- Own repo instead of a fork — forks restrict Actions permissions (GITHUB_TOKEN scope)
- Caught and fixed a tee pipe masking Semgrep's exit code — gate appeared green while findings existed

What I'd Do With More Time

- DAST (OWASP ZAP) against a running Juice Shop instance
- IaC scanning (Checkov/tfsec) if Terraform existed — plus a separate, non-CI-gated mechanism for catching things like AdministratorAccess IAM roles (a live account-state problem, not a code problem)
- Digest-pinning both Dockerfile base images for supply-chain integrity
- Full historical Gitleaks scan on onboarding each service (push-triggered scan only checks new commits)
- WAF/DDoS — explicitly out of scope for a CI/CD pipeline, belongs to a separate infrastructure workstream

End
